

Sharing Data with Databricks on AWS: A Decision Framework for 16 Patterns

by Manish Choudhary on 06 SEP 2026 in Databricks on AWS, Amazon S3, AWS Glue, Delta Sharing, Unity Catalog, Lakehouse Federation, Analytics | [Permalink](#) | [Comments](#) | [Share](#)

Organizations running Databricks on AWS routinely need to exchange data across teams, accounts, and platforms. A finance team wants a revenue dashboard, a partner wants a product feed, a data science team wants to build models on gold tables, and downstream systems on Snowflake, Power BI, or a custom application need governed access. Frequently the direction reverses, and the Databricks lakehouse must instead consume data from an external source.

The gap between "just share the data" and a safe, cost-efficient production rollout feels enormous — until you realize it comes down to a small set of decisions: whether the two sides share a Unity Catalog metastore, AWS account, and Region; the consumer platform; whether write access is required; the data classification; the freshness requirement; and, most fundamentally, whether you are the **producer** sharing data out or the **consumer** pulling data in. Get those right and the pattern is obvious. Miss any one and you are back to unnecessary cross-region egress charges, sensitive data on the wrong path, or brittle copy pipelines that are expensive to operate.

This guide walks through a decision framework that maps those variables to a recommended architecture, explains the reasoning behind each choice, and provides copy-paste implementation code for both the producer and consumer sides of 16 patterns. All examples use a fictitious setup (`demo_sales` catalog, `gold_products` schema) and illustrative placeholder names that you replace with your own.

Figure 1: Data sharing decision tree for Databricks on AWS

As we move through the framework, we show how each request maps to a pattern, who pays the compute, whether the data is copied or read in place, and how governance, security, and cost apply across all of them.

Section 1: Direction – Are You the Producer or the Consumer?

The single most useful distinction is **direction**. A **producer** exposes data to consumers wherever they run; a **consumer** brings external data into its own lakehouse. The two use different toolkits, so establishing direction first narrows the options quickly.

In the diagrams that follow, **PRODUCER** is the side sharing data out and **CONSUMER** is the side reading it in; a solid arrow (■■■■) is the control and metadata path (grants, catalog binding, protocol handshake), and the `s3://...` lane beneath it is the data path where the bytes reside. *Zero copy* means the consumer reads the producer's files in place rather than receiving a duplicate.

Two rules underpin every producer decision:

- **Delta Sharing is read-only.** The moment someone needs to *write*, you are no longer sharing — you are running shared storage with carefully scoped IAM roles.
- **JDBC/ODBC and the REST API bill the producer, not the consumer.** They run on your SQL Warehouse, so use Serverless with auto-stop and watch the compute. That is the price of zero-copy, always-fresh access.

Section 2: Produce – Which Pattern Shares Your Data Out?

When Databricks on AWS is sharing data out, the decision maps to one of nine patterns.

When...	Recommended architecture	Why
You need read-write access	Shared S3 External Location (IAM roles)	Delta Sharing is read-only, so writes force a shared-storage design
Both sides are in the same metastore	Direct GRANT + Cross-Catalog Views	Zero copy, zero latency, zero extra cost
Different metastores , both on Databricks	Databricks-to-Databricks (D2D) Sharing	Native catalog integration, no token juggling
Consumer is on Snowflake	Delta Sharing → Snowflake	Tables appear as native Snowflake external tables
Consumer wants native Iceberg (Snowflake, Trino, Flink, Athena)	Delta UniForm + UC Iceberg REST	One manifest serves any Iceberg engine — zero-copy, UC-governed
Consumer is on Power BI	Delta Sharing → Power BI	Import or DirectQuery via the built-in connector
Consumer uses Python / Spark / other	Delta Sharing (Open Protocol)	Works with any Delta Sharing client
Consumer is a BI tool or app needing live SQL	JDBC/ODBC via SQL Warehouse	Interactive queries through the standard driver — no extract to maintain
Consumer is an application calling over HTTP	SQL Statement Execution REST API	Programmatic access with no driver install

These patterns collapse into **two shapes**:

- **Bulk file access** — Delta Sharing (in all its flavors), Delta UniForm + Iceberg REST, and shared S3 hand the other side the underlying Parquet/Delta files, read via short-lived pre-signed URLs. The provider spends **zero compute**.
- **Live query** — JDBC/ODBC and the REST API run queries through a SQL Warehouse. The consumer always sees current data and needs no copy — but the **provider pays the compute** for every query.

There is also a choice of *protocol* for the bulk-file shape when the consumer is not on Databricks. **Delta Sharing** is Databricks' own protocol; **Delta UniForm + the Unity Catalog Iceberg REST catalog** publishes an Apache Iceberg metadata view over the *same* Parquet, so any Iceberg-native engine — Snowflake, Trino, Flink, Athena — reads it as a native Iceberg table.

Best for: Reach for UniForm when the consumer wants open-table-format interop or you want one manifest to serve many engines; reach for Delta Sharing when they are happy speaking the Delta Sharing protocol. Both are zero-copy and UC-governed.

Section 3: Consume – Which Pattern Brings Data In?

When your Databricks on AWS lakehouse is the consumer, you have five options. In nearly all cases, reading the data in place is preferable to copying it in.

You're receiving from...	Recommended architecture	Why
Another Databricks account/metastore	Mount the D2D share as a catalog	CREATE CATALOG ... USING SHARE — the data stays in the provider's S3, you query it live
Snowflake, Postgres, Redshift, MySQL, BigQuery	Lakehouse Federation	Register the source as a foreign catalog and query it in place — no ingestion pipeline
Another catalog — AWS Glue / Hive Metastore	Catalog Federation	Federate the whole metastore into UC as a foreign catalog; read Glue-registered tables via the three-level namespace
A partner's S3 bucket (Delta/Parquet files)	External Location + read-only IAM role	Register their path in Unity Catalog and read it directly
An open Delta Sharing provider	Delta Sharing client	Point Spark or delta-sharing at their .share profile

Federation is often the most valuable of these options. When a partner runs a Snowflake or PostgreSQL warehouse, the common instinct is to build an ingestion pipeline: schedule a job, land the data in bronze, reconcile schema drift, and operate it indefinitely. Federation avoids that entirely. You register the external database as a foreign catalog in Unity Catalog and query its tables live, in place, governed by the same permissions as your own tables.

Federation comes in two flavors:

- **Lakehouse Federation** connects to a *query engine* (Snowflake, Postgres, Redshift, BigQuery) over a JDBC-style CONNECTION and pushes queries down to it.
- **Catalog Federation** connects to a *metastore* — most commonly an **AWS Glue Data Catalog** or Hive Metastore — and surfaces every table it has registered under a single UC foreign catalog.

Best for: The Glue case is the common one on AWS — a "unified layer" where some tables are native UC Delta and others are Glue-registered S3 tables, all readable through the same `catalog.schema.table` namespace with governance enforced identically. The consumer never needs to know which physical reality backs a given table.

The consumer's default should be **"query in place," not "copy in."** D2D shares, Lakehouse Federation, and external-location reads all leave the data where it lives. You introduce a copy-based ingestion pipeline only when you need historical snapshots or when query volume makes live federation too chatty. Ingestion is the exception you justify — not the reflex you start with.

Section 4: Secure – How Do I Keep Sensitive Data on the Right Path?

Picking the pattern is only half the job. Turning a whiteboard box into a safe production rollout is the other half, and it is where most designs quietly go wrong.

Layer 1: Prerequisites and access as code

Do account-level groups exist yet? Is the consumer a service principal that needs OAuth credentials — and are you using OAuth machine-to-machine rather than a long-lived personal token, as recommended for automation? Is the consumer external, requiring a Unity Catalog RECIPIENT and a securely transmitted activation link?

In a governed estate, access is *as-code*: you open a Terraform PR to add the consumer's **service account** (never an individual) to the data product's consumer group; a compliance/classification gate and the domain owner's approval must pass before it goes live. And **verify the metastore relationship per consumer** — run `SELECT current_metastore()` on both sides rather than assuming from the account boundary. A consumer in a *different account* can still share your metastore (so it reads natively with a simple GRANT); two workspaces in the *same* account might not.

Layer 2: Environment matching

Prod talks to Prod, QA to QA, Dev to Dev. Delta Sharing recipients and IP allowlists are environment-scoped, and cross-environment access (Prod↔QA) is blocked by design — so a lower environment can never reach production data, whether over the sharing path or the JDBC fallback.

Layer 3: Security that scales with sensitivity

PII calls for dynamic views with column masking (`is_account_group_member`) and tagged PII columns. Confidential or restricted data adds audit logging, PrivateLink (no public internet), and a mandatory security-team review. For external recipients, harden the bearer token: set an expiration, rotate on a schedule, and restrict access with an IP allowlist so a leaked token is useless from an unknown network.

One hard line: PII + external consumer = high risk. Share only aggregated or anonymized data — never raw PII tables. When two parties genuinely need to compute on combined sensitive data, that is a job for a **Clean Room**, not a share.

Layer 4: Cost, before the invoice

Cross-region sharing incurs S3 egress — typically ~\$0.01–0.02/GB depending on the region pair; at high volume, replicate to the consumer's region instead of paying egress on every read. Remember which shape you picked: Delta Sharing reads cost the provider **zero compute**, while JDBC/ODBC and the REST API bill the provider's SQL Warehouse per query. For large API result sets, use the `EXTERNAL_LINKS` disposition (pre-signed URLs) rather than inlining megabytes of JSON.

Section 5: Implement – What Does the Code Look Like?

Once the pattern is chosen, the implementation is straightforward. For Databricks-to-Databricks sharing, the producer side is four statements:

```
-- PROVIDER SIDE
CREATE SHARE IF NOT EXISTS finance_share
COMMENT 'Data products shared with the Finance team';
ALTER SHARE finance_share ADD TABLE demo_sales.gold_products.revenue_dashboard;
CREATE RECIPIENT IF NOT EXISTS finance_team
USING ID '<CONSUMER_METASTORE_SHARING_ID>';
GRANT SELECT ON SHARE finance_share TO RECIPIENT finance_team;
```

The consumer mounts it as a catalog and queries it like any local table:

```
-- CONSUMER SIDE
CREATE CATALOG IF NOT EXISTS sales_shared
USING SHARE demo_provider.finance_share;
SELECT * FROM sales_shared.gold_products.revenue_dashboard LIMIT 10;
```

Federating a Snowflake source you consume is just as short:

```
-- CONSUMER SIDE – query Snowflake live, no ingestion
CREATE CONNECTION partner_snowflake TYPE snowflake
OPTIONS (host '<account>.snowflakecomputing.com', port '443',
user '<user>', password secret('scope','key'));
CREATE FOREIGN CATALOG partner_sales USING CONNECTION partner_snowflake
OPTIONS (database 'SALES_DB');
SELECT * FROM partner_sales.public.orders LIMIT 10; -- live, in place
```

Each producer pattern has an equally compact recipe — a JDBC URL, a parameterized REST API call, a Snowflake CATALOG INTEGRATION, a Power BI connector profile, or a `delta_sharing.load_as_pandas(...)` call. The full set of all 16 patterns, with an architecture diagram and both-sides code for each, is in the appendix.

Best for validating your choice: Four concrete scenarios show the framework in action:

- **Same metastore (producer)** → The sales team runs a workspace in `us-east-1`; the finance team shares the same metastore. Sales grants `SELECT`, Finance builds views. No data leaves the S3 bucket.
- **D2D sharing (producer)** → The central platform (account A, `us-east-1`) shares gold products with a subsidiary in account B, `us-west-2` — cross-region egress applies, so it is flagged up front.
- **REST API (producer)** → A customer-facing app on ECS/Lambda authenticates as a service principal via OAuth M2M and calls the Statement Execution API against a Serverless warehouse.
- **Lakehouse Federation (consumer)** → Analysts need a partner's Snowflake orders table. Instead of a nightly ingestion job, they register it as a foreign catalog and join it live against their own gold tables.

Section 6: Decide – The One-Page Cheat Sheet

#	Pattern	Direction	Zero-copy?	Who pays compute	Freshness	Governed by UC
1	Direct GRANT + views	Producer	Yes	consumer	live	Yes
2	D2D Sharing	Producer	Yes	consumer	live	Yes
3	Delta Sharing → Snowflake	Producer	Yes	consumer	on read	Yes
4	UniForm + Iceberg REST	Producer	Yes	consumer	async (REFRESH)	Yes
5	Delta Sharing → Power BI	Producer	Yes	consumer	refresh/live	Yes
6	Delta Sharing (open) → Python	Producer	Yes	consumer	on read	Yes
7	Shared S3 External Location	Producer	Yes (shared)	each side	live	Yes (both)
8	JDBC/ODBC via SQL Warehouse	Producer	n/a (live query)	producer	live	Yes
9	SQL Statement Execution API	Producer	n/a (live query)	producer	live	Yes
10	Lakehouse Federation	Consumer	Yes	source engine	live	Yes
11	Catalog Federation (Glue/Hive)	Consumer	Yes	consumer	live	Yes
12	External Location (partner S3)	Consumer	Yes	consumer	live	Yes
13	Consume open Delta Sharing	Consumer	Yes	consumer	on read	Yes

#	Pattern	Direction	Zero-copy?	Who pays compute	Freshness	Governed by UC
14	API Gateway / OData facade	Producer	n/a (live query)	producer	live	Yes
15	Provider PUSH → datamart	Producer	No (copy)	producer	batch	re-applied
16	Online Tables / Model Serving	Producer	n/a (serving)	producer	sub-second	Yes

Rules of thumb: Read-write → #7. Non-Databricks consumer who wants open table format → #4. Live BI/app → #8/#9 (you pay the compute). App over HTTP with no Databricks creds → #14 (gateway facade). Consumer *can't* pull → #15 (push, last resort). Sub-second per-row lookups → #16 (not a warehouse/sharing job). Pulling data in → **query in place (#10–12)** before you ever build ingestion.

Escalation ladder (zero-copy and UC-governed first): Direct GRANT ■ D2D Delta Sharing ■ Lakehouse/Catalog Federation · UniForm Iceberg REST ■ JDBC / API (+ gateway facade) ■ open Delta Sharing ■ **push/copy**. DEEP CLONE is a writable-snapshot pattern, not an access rung — it drops governance, so re-tag and re-mask after cloning.

Section 7: Troubleshoot – Why Doesn't It Work?

Most sharing "it doesn't work" tickets are one of a handful of recurring causes. Check the symptom before you deep-dive — it usually points straight at the fix.

Access and permissions

Symptom	Likely cause	Fix
PERMISSION_DENIED on SELECT	Group membership not synced via SCIM , or querying a non-output-port table	Confirm SCIM sync and that the service account is in the consumer group; only query gold output ports
403 on file read <i>despite</i> SELECT	Missing <code>EXTERNAL USE LOCATION</code> on the external location	Grant <code>EXTERNAL USE LOCATION</code> to the consumer group (excluded from <code>ALL PRIVILEGES</code>)
Grant looks correct but can't traverse	Missing a rung of <code>USE CATALOG</code> → <code>USE SCHEMA</code> → <code>SELECT</code>	Grant all three, to the group (never an individual), at the narrowest object
Recipient can't mount the share	Wrong sharing ID	Use <code>SELECT current_metastore()</code> (the metastore UUID, not a workspace/account ID)

Compute and connectivity

Symptom	Likely cause	Fix
SQL Warehouse unavailable	Warehouse stopped or not provisioned	Start it (auto-starts ~5–10s cold serverless)
JDBC times out from consumer	No network route (no PrivateLink)	Configure PrivateLink or VPC peering
401 / <code>invalid_client</code> on connect	Bad/expired SP secret or wrong OAuth URI	Rotate secret; verify <code>OAUTH_TOKEN_URI</code> ; use OAuth M2M
Cross-region read is slow	Different regions	Expect ~\$0.01–0.02/GB egress; replicate hot data or filter the share

Data correctness and governance

Symptom	Likely cause	Fix
Stale data after upstream change	UniForm metadata is async; no REFRESH	Run <code>ALTER ICEBERG TABLE ... REFRESH;</code> schedule to match SLA
No <code>iceberg.metadata.location</code>	UniForm not enabled, or deletion vectors ON	Set <code>enableIcebergCompatV2=true</code> , <code>enableDeletionVectors=false</code> ; OPTIMIZE
Masks not applied on consumer	Used open-protocol sharing or DEEP CLONE	Use D2D or Federation (both enforce masks), or pre-mask a view
PII appears where it shouldn't	Raw sensitive data on wrong path	Share only anonymized <code>contains_pii=false</code> gold; sensitive compute → Clean Rooms

Triage shortcut: Nine times out of ten it is one of three things — (1) **SCIM group membership** not synced, (2) **wrong environment/region/workspace**, or (3) a **warehouse that needs starting/provisioning**. Check those first.

Conclusion: Start Sharing

Choosing how to share data on Databricks on AWS is fundamentally a decision tree: a small set of inputs — direction, metastore relationship, consumer platform, access mode, data classification, and freshness — determines the right architecture. This guide made that tree explicit, walked through 16 producer and consumer patterns with implementation code for both sides, and covered the operational considerations (access as code, environment matching, governance propagation, and cost) that turn a chosen pattern into a safe production rollout.

To apply the framework:

1. Start from the decision tree in Figure 1 to identify your pattern.
2. Use the one-page cheat sheet to confirm its trade-offs.
3. Copy the corresponding provider and consumer code from the appendix, replacing the placeholder names with your own.

In every case, prefer the highest zero-copy, Unity Catalog–governed option before resorting to a copy-based pipeline.

To get started, explore [Delta Sharing](#) for open cross-platform sharing, [Lakehouse Federation](#) for query-in-place access, and [Unity Catalog](#) for governance across every pattern.

Appendix: All 16 Patterns — Both Sides, End to End

This appendix provides copy-paste recipes for every pattern. The running example uses Databricks on AWS (us-east-1, catalog `demo_sales`, schema `gold_products`) sharing with a finance team or an external partner. Replace the `<...>` placeholders with your own values.

Pattern 1 — Direct GRANT + Cross-Catalog Views (Same Metastore)

When to use: Producer and consumer live in the **same Unity Catalog metastore**. Simplest possible path — zero copy, zero cost.

Producer:

```
GRANT USE CATALOG ON CATALOG demo_sales TO `finance_team`;
GRANT USE SCHEMA ON SCHEMA demo_sales.gold_products TO `finance_team`;
GRANT SELECT ON SCHEMA demo_sales.gold_products TO `finance_team`;
```

Consumer:

```
CREATE CATALOG IF NOT EXISTS finance_catalog;
CREATE OR REPLACE VIEW finance_catalog.shared.revenue AS
SELECT * FROM demo_sales.gold_products.revenue_dashboard;
```

Same S3, live reads, zero cost. Use dynamic views (is_account_group_member) for masking.

Pattern 2 — Databricks-to-Databricks (D2D) Sharing (Different Metastores)

When to use: Both sides are on Databricks but in **different metastores/accounts**. Native, token-free, live.

Producer:

```
CREATE SHARE IF NOT EXISTS finance_share;
ALTER SHARE finance_share ADD TABLE demo_sales.gold_products.revenue_dashboard;
CREATE RECIPIENT IF NOT EXISTS subsidiary USING ID '<CONSUMER_METASTORE_SHARING_ID>';
GRANT SELECT ON SHARE finance_share TO RECIPIENT subsidiary;
```

Consumer:

```
SELECT current_metastore() AS my_sharing_id; -- send this to the provider
CREATE PROVIDER IF NOT EXISTS demo_provider USING ID '<PROVIDER_METASTORE_SHARING_ID>';
CREATE CATALOG IF NOT EXISTS sales_shared USING SHARE demo_provider.finance_share;
SELECT * FROM sales_shared.gold_products.revenue_dashboard LIMIT 10;
```

Metastore-to-metastore auth, no tokens. Cross-region means S3 egress (~\$0.01-0.02/GB). Add WITH HISTORY to share CDC.

Pattern 3 — Delta Sharing to Snowflake

When to use: The consumer runs **Snowflake** and speaks the Delta Sharing protocol (tables appear as external tables).

Producer:

```
CREATE SHARE IF NOT EXISTS snowflake_share;
ALTER SHARE snowflake_share ADD TABLE demo_sales.gold_products.revenue_dashboard;
CREATE RECIPIENT IF NOT EXISTS snowflake_partner;
GRANT SELECT ON SHARE snowflake_share TO RECIPIENT snowflake_partner;
DESCRIBE RECIPIENT snowflake_partner; -- returns the activation_link
```

Consumer (Snowflake):

```
CREATE OR REPLACE CATALOG INTEGRATION databricks_delta_share
CATALOG_SOURCE = DELTA_SHARING TABLE_FORMAT = DELTA ENABLED = TRUE;
CREATE DATABASE sales_from_databricks
FROM DELTA_SHARING_SHARE PROVIDER = 'databricks_provider' SHARE = 'snowflake_share';
SELECT * FROM sales_from_databricks.gold_products.revenue_dashboard;
```

Data stays in the provider's S3; Snowflake reads Parquet via pre-signed URLs. Harden the recipient with token expiration + IP allowlist.

Pattern 4 — Delta UniForm + Unity Catalog Iceberg REST to Any Iceberg Engine

When to use: The consumer wants a **native Apache Iceberg** table (Snowflake, Trino, Flink, Athena), or you want one manifest to serve many engines.

Producer — publish gold as Iceberg via UniForm:

```
ALTER TABLE demo_sales.gold_products.revenue_dashboard SET TBLPROPERTIES (
  'delta.enableDeletionVectors' = 'false',
  'delta.columnMapping.mode' = 'name',
  'delta.enableIcebergCompatV2' = 'true',
  'delta.universalFormat.enabledFormats' = 'iceberg');
OPTIMIZE demo_sales.gold_products.revenue_dashboard;
GRANT SELECT ON TABLE demo_sales.gold_products.revenue_dashboard TO `iceberg_readers`;
GRANT EXTERNAL USE SCHEMA ON SCHEMA demo_sales.gold_products TO `iceberg_readers`;
GRANT EXTERNAL USE LOCATION ON EXTERNAL LOCATION demo_gold_ext_loc TO `iceberg_readers`;
```

Consumer (Snowflake) — bind the UC Iceberg REST catalog:

```
CREATE CATALOG INTEGRATION uc_iceberg
  CATALOG_SOURCE = ICEBERG_REST TABLE_FORMAT = ICEBERG
  REST_CONFIG = (CATALOG_URI='https://<workspace-host>/api/2.1/unity-catalog/iceberg-rest',
  WAREHOUSE='demo_sales')
  REST_AUTHENTICATION = (TYPE=OAUTH, OAUTH_CLIENT_ID='<service-account-id>',
  OAUTH_CLIENT_SECRET='<secret>', OAUTH_TOKEN_URI='https://<workspace-host>/oidc/v1/token',
  OAUTH_ALLOWED_SCOPES=('all-apis')) ENABLED = TRUE;
CREATE ICEBERG TABLE revenue_dashboard
  CATALOG='uc_iceberg' EXTERNAL_VOLUME='uc_ext_vol' CATALOG_TABLE_NAME='revenue_dashboard';
ALTER ICEBERG TABLE revenue_dashboard REFRESH;
```

Same Parquet, read as native Iceberg — reusable by Trino/Flink/Athena too. Freshness is async: the consumer sees new rows only after REFRESH. EXTERNAL USE LOCATION is the key grant — it is excluded from ALL PRIVILEGES.

Pattern 5 — Delta Sharing to Power BI

When to use: The consumer is a **Power BI** team wanting dashboards over your gold tables.

Producer:

```
CREATE SHARE IF NOT EXISTS powerbi_share;
ALTER SHARE powerbi_share ADD TABLE demo_sales.gold_products.revenue_dashboard;
CREATE RECIPIENT IF NOT EXISTS powerbi_consumer;
GRANT SELECT ON SHARE powerbi_share TO RECIPIENT powerbi_consumer;
```

Consumer (Power BI Desktop): Download the .share profile from the activation link, then **Get Data** → **More...** → **Delta Sharing**, paste the profile URL, select revenue_dashboard, choose **Import** (scheduled refresh) or **DirectQuery** (live), publish and set a daily refresh.

Import = cheaper periodic snapshot; DirectQuery = live but heavier.

Pattern 6 — Delta Sharing (Open Protocol) to Python / Spark

When to use: The consumer works in **plain Python/Pandas** or **non-Databricks Spark** and has no Databricks account.

Producer:

```
CREATE SHARE IF NOT EXISTS ds_share;
ALTER SHARE ds_share ADD TABLE demo_sales.gold_products.customer_segments;
```

```
CREATE RECIPIENT IF NOT EXISTS ds_partner;
GRANT SELECT ON SHARE ds_share TO RECIPIENT ds_partner;
```

Consumer (Python):

```
import delta_sharing
PROFILE = "/path/to/config.share"
df = delta_sharing.load_as_pandas(
    f"{PROFILE}#ds_share.gold_products.customer_segments")
```

Provider spends zero compute; partner reads Parquet via pre-signed URLs — no Databricks account needed.

Pattern 7 — Shared S3 External Location (Read-Write)

When to use: The consumer needs to **write back** (enrich, round-trip). Delta Sharing is read-only, so this is the only path.

Producer — write side:

```
CREATE STORAGE CREDENTIAL provider_shared_cred
WITH (AWS_IAM_ROLE = 'arn:aws:iam::PROVIDER_ACCOUNT:role/write-role');
CREATE EXTERNAL LOCATION shared_exchange
URL 's3://data-exchange/sales-to-finance/'
WITH (STORAGE CREDENTIAL provider_shared_cred);
```

Consumer — read/write side:

```
CREATE STORAGE CREDENTIAL consumer_shared_cred
WITH (AWS_IAM_ROLE = 'arn:aws:iam::CONSUMER_ACCOUNT:role/partner-rw-role');
CREATE EXTERNAL LOCATION shared_exchange
URL 's3://data-exchange/sales-to-finance/'
WITH (STORAGE CREDENTIAL consumer_shared_cred);
CREATE TABLE partner_catalog.shared.revenue
USING DELTA LOCATION 's3://data-exchange/sales-to-finance/revenue';
```

The only path that supports writes. The S3 bucket policy is the real allowlist; each side uses its own IAM role.

Pattern 8 — JDBC/ODBC via SQL Warehouse (Live BI)

When to use: The consumer is a **BI tool or app** wanting live, interactive SQL through a standard driver — no extract to maintain.

Producer:

```
GRANT SELECT ON SCHEMA demo_sales.gold_products TO `finance_sp`;
-- Warehouse -> Permissions -> finance_sp = CAN USE; copy Server hostname + HTTP path
```

Consumer — JDBC URL:

```
jdbc:databricks://<host>:443;httpPath=/sql/1.0/warehouses/<id>;SSL=1;AuthMech=11;Auth_Flow=1
```

Unity Catalog enforces grants per query. Provider pays warehouse compute — use Serverless + auto-stop.

Pattern 9 — SQL Statement Execution REST API (App Integration)

When to use: The consumer is an **application** that calls over HTTPS and should not install a driver.

Producer:

```
GRANT SELECT ON SCHEMA demo_sales.gold_products TO `app_sp`;
-- Warehouse -> Permissions -> app_sp = CAN USE; note the warehouse_id
```

Consumer (Python over HTTPS):

```
resp = requests.post(f"{HOST}/api/2.0/sql/statements",
headers={'Authorization': f'Bearer {TOKEN}'},
json={'warehouse_id': WH, 'catalog': 'demo_sales', 'schema': 'gold_products',
'statement': 'SELECT * FROM revenue_dashboard WHERE region = :region LIMIT 100',
'parameters': [{'name': 'region', 'value': 'us-east-1', 'type': 'STRING'}],
'disposition': 'INLINE'})
```

Parameterized statements prevent injection; use EXTERNAL_LINKS for large exports.

Pattern 10 — Lakehouse Federation (You Are the Consumer)

When to use: You need a **query engine's** data (Snowflake, Postgres, Redshift, BigQuery) live, without building an ingestion pipeline.

Consumer (Databricks):

```
CREATE CONNECTION partner_snowflake TYPE snowflake OPTIONS (
host 'partner_account.snowflakecomputing.com', port '443',
user secret('partner','sf_user'), password secret('partner','sf_password'));
CREATE FOREIGN CATALOG partner_sales USING CONNECTION partner_snowflake
OPTIONS (database 'SALES_DB');
SELECT o.order_id, o.amount, g.margin
FROM partner_sales.public.orders o
JOIN demo_sales.gold_products.product_catalog_public g USING (product_id);
```

Queries push down to Snowflake — no copy into your S3. Materialize locally only if it gets chatty.

Pattern 11 — Catalog Federation: AWS Glue / Hive Metastore (You Are the Consumer)

When to use: You need to read a whole **AWS Glue / Hive Metastore** — a "unified layer" mixing native UC Delta and Glue-registered S3 tables — through one namespace.

Setup — federate the Glue catalog once:

```
CREATE CONNECTION glue_conn TYPE glue OPTIONS (
aws_region 'us-east-1', aws_account_id '<glue-account-id>',
credential '<uc-service-credential-id>');
CREATE FOREIGN CATALOG glue_federated USING CONNECTION glue_conn
OPTIONS (storage_root 's3://demo-glue-backed/');
```

Consumer — read Glue-registered tables via the three-level namespace:

```
df = spark.read.table("glue_federated.curated_sales.fct_order_line")
df.filter("order_date >= '2026-07-01']").show(5)
```

The consumer never needs to know a table lives in Glue-backed S3 vs. UC-managed storage — UC routes and governs both.

Pattern 12 — Consume a Partner's S3 Bucket (You Are the Consumer)

When to use: A partner drops **Delta/Parquet files** in their **S3 bucket** and grants you a read-only cross-account IAM role.

Consumer (Databricks):

```
CREATE STORAGE CREDENTIAL partner_read_cred
WITH (AWS_IAM_ROLE = 'arn:aws:iam::PARTNER_ACCOUNT:role/read-role');
CREATE EXTERNAL LOCATION partner_feed
URL 's3://partner-feed/demo-exports/' WITH (STORAGE CREDENTIAL partner_read_cred);
```

```
CREATE TABLE bronze.partner_orders
USING DELTA LOCATION 's3://partner-feed/demo-exports/orders';
```

Read-in-place = no duplicate storage; COPY INTO instead for an isolated local snapshot.

Pattern 13 — Consume an Open Delta Sharing Feed (You Are the Consumer)

When to use: You subscribe to a **data vendor's open Delta Sharing feed** (the vendor may not even be on Databricks).

Consumer (Databricks):

```
PROFILE = "/Volumes/main/secure/config.share"
df = spark.read.format('deltaSharing').load(f"{PROFILE}#vendor_share.gold.metrics")
df.write.mode('overwrite').saveAsTable('bronze.vendor_metrics')
```

Works even if the vendor is not on Databricks. Persist into bronze only if you need to keep it.

Pattern 14 — API Gateway / OData Facade to SQL Statement Execution

When to use: An external app must read over **plain HTTPS** and **must not hold Databricks credentials**. A managed gateway fronts the Statement Execution API.

Producer — grant the backend service principal:

```
GRANT USE CATALOG ON CATALOG demo_sales TO `app_backend_sp`;
GRANT USE SCHEMA ON SCHEMA demo_sales.gold_products TO `app_backend_sp`;
GRANT SELECT ON SCHEMA demo_sales.gold_products TO `app_backend_sp`;
```

Consumer (App) — talks only to the gateway:

```
import os, requests
GATEWAY = os.environ['DATA_API_GATEWAY']
resp = requests.get(f"{GATEWAY}/gold_products/revenue_dashboard",
headers={'x-api-key': os.environ['DATA_API_KEY']},
params={'$select': 'region,revenue', '$filter': "region eq 'us-east-1'"})
```

Same governed source as Pattern 9 — the difference is the app never sees Databricks credentials. Gold + contains_pii=false only.

Pattern 15 — Provider PUSH to Consumer Datamart (Last Resort)

When to use: The consumer **cannot pull** (a legacy app / relational datamart). The provider pushes governed gold on a schedule. Copy-based — use only when no pull path exists.

Producer — scheduled job:

```
from pyspark.sql import functions as F
gold = (spark.read.table("demo_sales.gold_products.revenue_dashboard")
.filter(F.col("updated_at") > last_watermark))
(gold.write.format("jdbc")
.option("url", DATAMART_JDBC_URL)
.option("dbtable", "dbo.stg_revenue_dashboard")
.option("truncate", "true").mode("overwrite").options(**JDBC_PROPS).save())
# Then server-side: MERGE stg INTO target ON PK; advance watermark in same txn
```

The last rung of the ladder. Physical copy + batch latency, and the provider owns the DBU. Because the data leaves Unity Catalog, mask at source and re-tag downstream.

Pattern 16 — Online Tables / Model Serving (Sub-Second)

When to use: **Sub-second, per-row** lookups for live inference. This is **not** a SQL Warehouse or Delta Sharing job.

Producer:

```
CREATE TABLE demo_sales.gold_products.customer_features AS SELECT ...;
```

If your "real-time" need is single-row inference, reach for serving — not a warehouse. Anonymized gold only for external/ML serving.